ALP/CNL is the workshop's mouth. LexLean is the first mirror. Everything downstream — PrismPM viewpoints, Hologram archives, Foundry sites — is only as closed as this joint. A civic claim that cannot pass it is still conversation, not infrastructure.

What the public contract actually names

LexLean's own specification calls the authoring language a closed-lexicon controlled natural language. The acronym CNL is in the contract. The acronym ALP is not.

What the repos do publish under that stem is `alp_sorry_manifest.json` in PhaseMirror/Foundry: an honesty ledger for transitional `sorry` s in the Lean scaffold, governed as admitted proof debt, not as a language. On the current main snapshot that ledger is an empty schema-2.0 list. Older views of the same file carried amnesty entries (ADR-PML-005, examiner-governed, deadline 2027-06-30). That object is the admission boundary for unfinished kernel text. It is not the authoring face.

So the pair, stated without inventing a repository, is:

- CNL: the closed language a person writes.
- ALP, as currently evidenced: the honesty gate on Lean that has not yet closed. A sentence may enter the workshop only if its proof debt is either absent or explicitly manifested. Silent `sorry` is not a Foundry claim.

The compiler that binds them is LexLean, normative contract `LEXLEAN-SPEC-1` .

The authoring contract

A Foundry author does not write Lean, and does not write free LaTeX. The only source is a `.lex.tex` module: UTF-8 NFC, LF-only, no tabs, no TeX comments, no raw macros. The document is a program. Human-visible mathematical text and kernel-visible Lean declarations are two renderings of one linked semantic object.

The SPEC's central property is total coverage:

> *No accepted LexLean document contains an uninterpreted word, symbol, punctuation mark, control sequence, reference, or semantically relevant structural token.*

That coverage is four closures, not a style guide.

Closure	What it refuses

Lexical	Any atom not declared by exactly one lexicon origin (`LX-01` – `LX-14`). Ambiguity is rejected; no priority heuristic picks a winner (`LX-13`).
Symbolic	A glyph or control not in the core renderer-token registry. Non-core lexicons cannot inject raw TeX (`GL-08` , `TX-07`).
Grammatical	A sentence that does not parse under the fixed structural, mathematical, and proposition grammar (`GR-01` – `GR-16`). Packages cannot add productions (`GL-04`).
Semantic	A denotation that is not core, Lean, document, or an acyclic defined value (`GL-05`). Cycles in imports, defined denotations, or document definitions are rejected.

The LaTeX backend must not copy an author’s sentence, alias spelling, whitespace, TeX control, or punctuation into the output. Canonical text is re-rendered from the IR through renderer tokens. The Lean backend must not emit comments, strings-as-prose, holes, or author axioms. Neither backend may accept an opaque bypass.

That is the phase mirror at L0: the workshop speaks once, and both the page and the kernel are reflections. If the two faces diverge, the source was never closed.

The lexicon is the civic dictionary

A lexicon package is a versioned, locked vocabulary, not a glossary appendix. Layout is closed: `lexicon.toml` plus entry files. Each entry has one category, one fixed frame, surface forms (channel, inflection, canonical spelling), a denotation, an LSE signature (scoped binders and universes), and an LRE render expression. Imports are exact. A package whose language differs from the project language is rejected at lock time.

The author names a package in the module header:

```latex
```

```
\begin{lexlean}{Main}
\useglossary{lexlean.std.nat@1.0.0}
\title{Natural number addition}

\begin{theorem}{add-zero}
\noaxioms
For every natural number \(n\), \(n + 0 = n\).
\begin{proof}
Close the goal by reflexivity.
\end{proof}
\end{theorem}
\end{lexlean}
```

That block is the complete SPEC §29 document. “For every,” “natural number,” n, $+$, 0, $=$, and “Close the goal by reflexivity” are not English. They are lexicon forms with pinned denotations. Reflexivity lowers to `rfl`. The theorem header lowers to a public Lean theorem. `\noaxioms` is a policy, not a slogan.

Built-ins that matter for the Foundation:

- `lexlean.core`, `lexlean.std.nat@1.0.0`, `lexlean.std.int@1.0.0` — the arithmetic floor.
- `lexlean.uor.atlas@1.0.0` — locked into every project and unconditionally visible, whether or not the document names an Atlas entry. Its visibility closes over core, nat, and int. From 0.1.1 the UOR Atlas is the foundation model of LexLean: every accepted document carries the Atlas-derived header, and the Atlas formalization is closed inside the repository (5,519 environment declarations from one native source, no second implementation).

A Foundry workshop lexicon — roles, sites, controls, workflows — is the same kind of object. It may add entries. It may not add grammar productions, raw TeX, or a denotation cycle. New civic vocabulary is admitted by package and lock, not by prose.

One IR, two faces

Parse does not emit either backend. It elaborates a typed, linked intermediate representation: document IR, term IR, proof IR, and `GlobalRef`s that point at core, external, document, or defined lexicon entries. The semantic snapshot is the canonical serialization of that IR (`lexlean/semantic-snapshot/1`). It carries every closed declaration, term, proof variant, and axiom policy, and it carries no host paths and no backend text.

Language versions are not a migration convenience:

- 1.0 is the prose-structured document language. Definitions, theorems, and structured proofs. Compiler identity is reported as language 1.0 even in the 0.3.0 package line.
- 1.1 is the immutable PrismPM integration line. It adds the closed portable data and operation vocabulary, plus generic semantic declarations: structures, classes, instances, finite inductives, total structural recursion, exhaustive matches, Boolean validators, exact theorem application, axiom-free Boolean reflection. The semantic-1.1 fixture contains no handwritten Lean.
- 1.2 extends the snapshot with products, first-class functions, ordered collections, graphs, and recursion evidence. A 1.1 lock is not silently upgraded.

PrismPM authors in 1.1. That is why Foundry models are `.lex.tex` graphs rather than Lean files: the snapshot is the object PrismPM is allowed to project.

Proofs are sentences with a policy

The proof language is pinned to Lean lowerings (`PF-01` – `PF-19`), including closed linear arithmetic. “Assume,” “Apply,” “constructor,” “cases,” “induction,” “rewrite,” and “Close the goal by reflexivity” are productions, not tactics the author invents.

Every declaration carries one explicit axiom policy, audited by parsing `#print axioms` :

- `\noaxioms` — observed set empty.
- `\allowaxioms{…}` — observed set is a subset.
- `\exactaxioms{…}` — observed set is exactly that set.

There is no default inheritance. An imported theorem’s axioms fall under the declaring component’s policy. Author-defined axioms and opaque declarations are outside the language. A Foundry claim about a site, a control, or a workflow is therefore not “we proved it” but “this declaration’s observed axiom set matches its declared policy.”

The loop, and what each step is allowed to say

```text
```

```
lexlean init   → project skeleton, lexlean.toml
lexlean lock   → canonical lock: compiler-semantics digest, package closure
lexlean check  → lexical, grammatical, semantic validation; no verification
claim
lexlean build  → linked IR, canonical LaTeX, prose-free Lean, maps,
manifest
lexlean verify → pinned Lean elaboration, leanchecker replay, axiom audit,
attestation
```

Honesty level on the conformance register is `build` : each ID is constructed in-repo and checked by `conformance_<id>` . Verified status is claimed by `verify` alone (`VR-18`). `leanchecker` is a same-kernel replay, not an independent checker (§22.4). The pin is `leanprover/lean4:v4.32.1` . Semantic IDs are platform-independent and include linked IR, lexicon closure, language semantics, and toolchain ID.

Current package identity on the verification record is lexlean 0.3.0, language 1.0, with a compiler-semantics digest and that same Lean pin. Language 1.1 rides as the extension PrismPM consumes; it does not weaken the 1.0 full-spec gate.

Why this is the workshop’s L0 gate

Phase-mirror methodology, at this joint, is not a metaphor. One semantic object is reflected into a page a citizen can read and a kernel term nothing can silently extend. The reflection is contractive in the only sense this layer can enforce: no new atoms, no new productions, no heuristic parse, no second Atlas, no handwritten Lean on the way down.

A sentence that cannot lower is not yet a Foundry claim. It has failed lexical, grammatical, or semantic closure, or it has failed its axiom policy. It may remain in the workshop as draft. It may not become a Prism model, a Hologram guest, or a site workflow. The empty `alp_sorry_manifest` is the matching discipline on the other side of the mirror: unfinished kernel text is either gone, or named, governed, and dated. Unnamed proof debt does not enter the base-state.